

Universidad ORT Uruguay
Facultad de Ingeniería

Estructuras de Datos y Algoritmos 2

Obligatorio 1

Facundo García - 328757
Juan Martín Lasserre – 289139

Tutores: Juan Pedro Puig y Francisco Fernández

Índice

Declaración de autoría	3
Ejercicio 1: Sistema de Rankings de Jugadores	4
Ejercicio 2: Sistema de Caché de Recursos Web (hashing cerrado doble)	5
Ejercicio 3: Sistema de Caché de Recursos Web (hashing abierto)	8
Ejercicio 4: Fusión de K Listas Ordenadas	10
Ejercicio 5: Árbol de Cubrimiento Mínimo	11
Tabla de resultados	12

Declaración de autoría

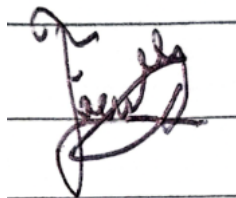
Nosotros, Facundo García, y Juan Martín Lasserre, declaramos que el trabajo que se presenta en esta obra es de nuestra propia mano. Podemos asegurar que:

- La obra fue producida en su totalidad mientras realizábamos el Obligatorio 1 de Estructuras de Datos y Algoritmos 2;
- Cuando hemos consultado el trabajo publicado por otros, lo hemos atribuido con claridad;
- Cuando hemos citado obras de otros, hemos indicado las fuentes. Con excepción de estas citas, la obra es enteramente nuestra;
- En la obra, hemos acusado recibo de las ayudas recibidas;
- Cuando la obra se basa en trabajo realizado conjuntamente con otros, hemos explicado claramente qué fue contribuido por otros, y qué fue contribuido por nosotros;
- Ninguna parte de este trabajo ha sido publicada previamente a su entrega, excepto donde se han realizado las aclaraciones correspondientes.



Juan Martín Lasserre

23/10/2025



Facundo García

23/10/2025

Ejercicio 1: Sistema de Rankings de Jugadores

Para la resolución del ejercicio 1 utilizamos mayormente las diapositivas del curso. (Agregar nodo junto con las rotaciones). Utilizamos un AVL para los puntajes y otro para los jugadores para poder garantizar los órdenes que nos pedía el problema.

ADD

Utilizamos el agregarNodo en un AVL, agregando el jugador y su puntaje correspondiente. Luego actualizamos la variable puntajeMax y top1 solo si es necesario. Al ser una operación de inserción regular en un AVL, cumple con ser $O(\log n)$.

Utilizamos el código visto en las diapositivas del curso de agregarNodo en un AVL, como así también las rotaciones simples y dobles. Tuvimos que usar una función “wrapper” para prevenir el acceso directo al `NodoAVL* raiz`.

FIND

Hicimos una función encontrarJugador, que es la función buscar en ABB según su ID. El peor caso es que el nodo a encontrar sea una hoja, empezando desde la raíz, ese caso tiene $O(\log N)$. Hicimos una búsqueda en ABB, vista en Estructuras de datos y algoritmos 1. Nuevamente usamos un wrapper.

RANK

Para resolver este problema debimos ir llevando la cuenta de la altura de cada subárbol de todo nodo. Originalmente habíamos pensado recorrer el árbol entero y contar cuántos cumplían con la condición, pero esto claramente era $O(n)$. Para cada inserción, fuimos llevando la cuenta de cuántos nodos había en sus dos subárboles. Hicimos una función auxiliar puntajeSuperiorA, que mira el nodo donde estoy parado y compara, si es mayor al puntaje recibido suma todos sus hijos derechos (son mayores, no hace falta buscar), suma el nodo que estoy junto con sus repeticiones y busca en hijos izquierdos (puede haber algún menor a ese nodo que sí lo cumpla). Si el puntaje es menor busca en los hijos derechos (mayores). Y si el puntaje es igual suma todos los de la derecha y suma el nodo que estoy junto a los hijos derechos. Esta operación se hace recursivamente.

Tiene $O(\log N)$, no recorre todos los casos. (está explicado anteriormente, se especifican todos los casos). Primer caso busca solo en la izquierda, segundo caso busca solo en la derecha, último caso no busca, solo agrega. El peor caso es que

repetidamente deba revisar el subárbol izquierdo, después el derecho del izquierdo y así sucesivamente hasta llegar a una hoja, descartando un subárbol en cada bajada, dando así un peor caso de $O(\log n)$. No tuvimos referencia de internet, ni de trabajo hecho en clase, solo fuimos pensando que pasaría en cada caso haciendo un dibujo y desarrollando el código.

TOP1

Como debía ser en orden 1, hicimos un atributo que sea top1, se actualiza con cada inserción como corresponda (si el puntaje no fue mayor, no se actualiza). Lo devolvemos en $O(1)$ como pedía el problema.

COUNT

Como debía ser en orden 1, hicimos un atributo que sea cantidad, se actualiza con cada inserción. Devolvemos este atributo. Tiene $O(1)$, solo devuelve el atributo. Como la función tenía que tener $O(1)$ no nos quedaba otra opción que agregarlo como atributo.

Ejercicio 2: Sistema de Caché de Recursos Web (hashing cerrado doble)

Para este ejercicio, debimos utilizar dos tablas de hashing cerrado doble: una para poder acceder rápidamente a los recursos (dominio_path) y garantizar $O(1)$ para CONTAINS y otra con únicamente los dominios (dominios), permitiéndonos realizar operaciones como LIST_DOMAIN y COUNT_DOMAIN en el orden solicitado. Obtuvimos las dos funciones de hash de internet (los links están como comentarios en el código), utilizando la metodología de cálculo explicitada en las presentaciones del curso (específicamente, <https://hash-table.uruguayan.ninja/16?clicks=4>). Como se mantenían posiciones fijas para cada elemento de la tabla de recursos, a diferencia del ejercicio 3, preferimos que cada dominio fuera registrando las posiciones de cada uno de sus recursos en vez de duplicarlo y almacenarlo como una lista. Esto podría haber simplificado las operaciones, pero también habría duplicado el espacio utilizado. Sí utilizamos dicho procedimiento para el ejercicio 3, como se detalla más adelante.

PUT

Se inserta en dominio_path en $O(1)$ usando dominio+path como clave. No hacemos distinción si se actualizó un nodo o se agregó. Acto seguido, procedemos

a buscar el dominio en la tabla dominios. Si no existía, lo creamos. Si ya existía, debemos recorrer toda la lista de índices de ese dominio para actualizar los datos del recurso, si es que ya existía, ciclando a través de la lista. Necesariamente debemos recorrer toda la lista; los nodos nuevos o actualizados deben agregarse al principio, manteniendo un orden descendente de actualización/inserción. Por último, insertamos el nodo nuevo o actualizado y retornamos en $O(1)$. Esto nos da una complejidad de $O(k)$, donde k es la cantidad de elementos del dominio solicitado.

No contábamos con información sobre la implementación de esta función, así que debimos investigar en Internet. Aplicamos un razonamiento similar con un `while` para las demás funciones que necesitaran buscar espacios libres. Sí vimos en clase que un factor de carga recomendado era 0.7, así que en la verificación antes de insertar utilizamos ese valor.

GET

Esta función ejecuta una búsqueda con el mismo bucle `while` de PUT pero en `dominio_path`, que almacena en cada entrada un recurso en particular. Como es una operación de búsqueda en tabla de hash y las dos funciones de hash elegidas son buenas, tiene orden $O(1)$ promedio. La implementamos sin documentación ni uso de IA, utilizando un razonamiento similar al de PUT para hallar la siguiente posición.

REMOVE

REMOVE es muy parecida a PUT en que ambas deben recorrer todos los elementos del dominio del recurso para eliminarlo. Además deben necesariamente recorrerlo entero para mantener constante el orden de inserción/actualización, por lo que necesariamente queda limitado a $O(k)$ siendo k la cantidad de elementos en el dominio. Como insertar en `dominio_path` es $O(1)$ promedio, no nos afecta. Nuevamente, no utilizamos bibliografía del curso ni de internet: lo pensamos a partir del funcionamiento de PUT.

CONTAINS

CONTAINS es una búsqueda en tabla de hash común y corriente. Como nos dan el dominio y el path, es mucho más rápido buscarlo en `dominio_path`, garantizando $O(1)$ promedio.

COUNT_DOMAIN

Esta función de cierta forma también es una búsqueda, solo que queremos retornar un atributo si es que existe. Buscamos el dominio ingresado en dominios en $O(1)$ promedio: si existe, mediante una función auxiliar, retornamos un puntero a su lista de índices (aquí guarda las posiciones en dominio_path de sus recursos). Si dicha lista es nullptr, imprimimos 0. De lo contrario, imprimimos la cantidad de elementos que tenga. Cabe aclarar que implementamos esta lista de índices mediante el TAD ListImp, incluido en la carpeta tads.

LIST_DOMAIN

Su funcionamiento es muy similar a COUNT_DOMAIN: obtenemos todos los recursos asociados a ese dominio y llamamos a mostrarPaths, quien los imprime uno a uno. Se obtiene la lista de índices en $O(1)$ promedio y como la recorremos entera, obtenemos un orden total de $O(k)$ promedio, influenciado por qué tan rápido se pueda obtener la lista y siendo k la cantidad de recursos del dominio.

CLEAR_DOMAIN

Volvemos a utilizar una función auxiliar para recuperar un puntero a la lista de índices de un dominio en $O(1)$ promedio. Para cada índice, lo eliminamos de dominio_path y dominios, ambas en $O(1)$ promedio. Recorremos todos los índices del dominio una sola vez, dándonos así un orden de $O(k)$ promedio, con k siendo la cantidad de recursos del dominio a limpiar.

SIZE

Para implementar esta función, fuimos llevando registro de la cantidad de recursos únicos del sistema mediante un contador en dominio_path: basta con retornar dicha cantidad para garantizar $O(1)$ peor caso.

CLEAR

Para cada elemento de dominios, si hay un dominio, hacer CLEAR_DOMAIN de ese dominio. Esto nos da un orden $O(M+K)$ donde M es el largo de la tabla (recorremos toda la tabla) y K es el total de los elementos del recurso. Como CLEAR_DOMAIN se ejecuta en $O(k)$ promedio, siendo k la cantidad de elementos en el dominio a limpiar, K se podría expresar como la sumatoria de las cantidades de recursos de cada dominio.

Ejercicio 3: Sistema de Caché de Recursos Web (hashing abierto)

Este ejercicio fue muy similar al anterior, solo que obviamente utilizando otra estrategia de hashing. Explotamos al máximo el TAD ListImp para facilitar las operaciones de inserción y eliminación. Asimismo, reutilizamos una de las funciones de hash del ejercicio anterior, referenciada adecuadamente en el código. Volvimos a utilizar dominios y dominios_path, pero con una diferencia marcada: mientras que dominios_path contiene listas de NodoHash (la estructura que utilizamos para almacenar cada recurso), dominios es una tabla de hash de listas de NodoDominio. Esta estructura adicional nos permitió poder acceder rápidamente a cada dominio y consecuentemente a sus recursos. Para dicho propósito, cuenta con una lista de NodoHash, que esencialmente duplica el nodo. Intentamos hacerlo con los índices de cada elemento, guardando la posición en dominios_path y su índice en el bucket, pero nos enfrentamos al problema de eliminar. Esta operación requería necesariamente recorrer todo el bucket y actualizar los índices de todos los elementos dependiendo de la posición del recurso eliminado. Como no teníamos restricciones de espacio, optamos por la opción más sencilla de entender e implementar, aunque claramente más ineficiente.

No utilizamos mucha documentación del curso ni de Internet: fuimos implementando todo en base a lo que ya sabíamos de Estructuras de datos y algoritmos 1, donde ya habíamos trabajado a profundidad con tablas de hash abiertas.

PUT

En vez de buscar por alguna casilla libre como en el hashing cerrado, en hashing abierto es mucho más sencillo: calculamos una única posición e insertamos en el bucket indicado. Dependiendo de si el bucket ya tenía elementos o no, lo recorremos en caso de que se trate de una actualización. En general, como los buckets nunca estarán muy llenos, esta operación se lleva a cabo en $O(1)$ promedio. Asimismo, incluimos una verificación del factor de carga para comprobar si es posible la inserción o no. Debemos hacerla tras recorrer todo el bucket pues se puede tratar de una actualización, en cuyo caso la operación debería ser válida incluso si el factor de carga no es lo suficientemente bajo.

Tras insertar en dominios_path, dependiendo de su resultado, insertamos en dominios, siguiendo una lógica más o menos similar: buscar un bucket para el dominio: si ya existe, insertar el nodo en su lista de recursos. Si no existía, crear uno nuevo e insertar el nodo. La operación de inserción en NodoDominio se encarga de actualizar la posición del recurso, recorriendo toda la lista de recursos, por lo que su complejidad total termina siendo de $O(k)$ promedio, dominado por la recorrida sobre los recursos del dominio.

GET

Esta es una función de búsqueda normal: calculamos la posición del recurso y lo buscamos en el bucket indicado en `dominio_path`. Si lo encontramos, imprimimos su título y tiempo: de lo contrario, imprimimos “recurso_no_encontrado” y salimos. En este caso no utilizamos el valor booleano que devuelve la función auxiliar `get()`. Se ejecuta en $O(1)$ promedio.

REMOVE

Calculamos la posición del recurso y lo buscamos en `dominio_path`. Si no lo encontramos, retornamos `false`. Si está, lo removemos del bucket y retornamos `true`. A partir de este valor decidimos si es necesario eliminarlo de la lista de recursos de ese dominio: esta operación se realiza en $O(k)$ promedio, con k siendo la cantidad de recursos del dominio, porque debemos ir a buscarlo a la lista de recursos de ese `NodoDominio` en particular.

CONTAINS

Al contar con una función auxiliar `get()` y la tabla de hash únicamente con los recursos, esta operación resultó muy sencilla en nuestra implementación. Calculamos la que debería ser su posición en `dominio_path`, recorremos el bucket y retornamos `true` si existe, `false` en el caso contrario. En promedio, esta operación se procesaría en $O(1)$, pudiendo empeorar si la carga de los buckets no se distribuye equitativamente.

COUNT_DOMAIN

Busco un dominio en `dominios`: si lo encuentro, devuelvo el largo de su lista de recursos. Si no lo encuentro, devuelvo 0. Esto es $O(1)$ promedio, dominado por la búsqueda del dominio en su bucket.

LIST_DOMAIN

Busco un dominio en `dominios`: si lo encuentro, llamo al método `listarRecursos()` dentro del class `NodoDominio`. Si no lo encuentro, imprimo una línea vacía, como pide la letra. `listarRecursos()` simplemente recorre los recursos de un dominio e imprime sus paths. Como recorre la lista entera y recuperar dicho dominio cuesta $O(1)$ promedio, la operación total termina siendo de complejidad $O(k)$ promedio, con k siendo la cantidad de recursos de ese dominio.

CLEAR_DOMAIN

Recuperamos una lista con todos los recursos de un dominio en $O(1)$ promedio, pues primero debemos buscarlo en dominios en $O(1)$ promedio. Hasta que la lista esté vacía, recuperamos el primer elemento de la lista en $O(1)$ peor caso, lo eliminamos de `dominio_path` en $O(1)$ promedio y lo quitamos de la lista recuperada en $O(1)$ peor caso. Después de esto, lo eliminamos en dominios en $O(1)$ caso promedio. En total, la operación tiene una complejidad de $O(k)$ promedio, dominada por la búsqueda inicial del dominio y el tener que recuperar todos sus paths.

SIZE

Idéntica al ejercicio anterior: recupera en $O(1)$ peor caso la cantidad de elementos de `dominios_path` y la imprime.

CLEAR

Para cada elemento de la tabla dominios, recuperamos el bucket con los dominios. Para cada uno de ellos, ejecutamos `CLEAR_DOMAIN` y lo eliminamos de dominios en $O(1)$ promedio. En total, cumplimos con la complejidad $O(M+K)$ pedida: recorreremos una sola vez toda la tabla y eliminamos todos los recursos del sistema.

Ejercicio 4: Fusión de K Listas Ordenadas

Utilizamos la implementación de `minHeap` que vimos en el curso. Con la excepción del insertar, que le agregamos como parámetro `lista Origen` para ver de qué lista viene. Con el `minHeap` hecho:

- Guardamos k (cantidad de listas) y para cada lista guardamos N_i (largo de la lista). Cargamos la lista con los valores que me van pasando.
- Creamos un array de índices, para ver en qué posición estoy en cada lista. Lo inicializo en 0.
- Creamos un heap con capacidad para k elementos. Y recorreremos, si la lista i tiene elementos, inserto el primer elemento de la lista en el heap, y avanzo uno en el índice de esa lista.
- Mientras el heap tenga elementos, agarro el primer elemento del heap (el menor), lo imprimo y lo elimino del heap, si esa lista sigue teniendo elementos lo agrego al heap.

El heap siempre tiene un máximo de k elementos, uno de cada lista, y va imprimiendo el primero (menor elemento). Cuando el heap se queda sin elementos es porque imprimió todos los valores.

La función cumple con que el heap tenga $O(K)$ de espacio, el heap nunca tiene más de k elementos, solo tiene uno de cada lista a la vez, cuando elimino uno agrego otro.

Se recorre n veces el while y dentro del while hace secuencialmente `eliminarTope()` e `insertar()`, que tienen $O(\log K)$. Por lo que cumple con $O(N \log K)$.

Utilizamos chatGPT, que nos inclinó a cambiar la función `insertar`, para almacenar la lista de origen del valor insertado, usado posteriormente en la función `main`. No fue código generado, nos dio la idea y nosotros la implementamos.

Ejercicio 5: Árbol de Cubrimiento Mínimo

Utilizamos el `minHeap` de aristas, es muy parecido al `minHeap` de enteros que hicimos en clase. También sacamos de las diapositivas del curso la clase de `conjuntosDisjuntos v2`, para optimizar la función de `unir()`.

Por último creamos la clase `Grafo`, también vista en el curso.

Usamos la función `Kruskal` que está en las diapositivas del curso.

`Kruskal` tiene $O((V+E) \log V)$, que es $O(E \log V)$, si el grafo es denso.

Utilizamos IA para verificar el orden, ya que en las diapositivas nos decía que `Kruskal` tiene $O(E \log V)$, y el ejercicio debía tener $O((V+E) \log V)$. Como recorremos todo el heap de aristas y vamos removiendo del heap en $O(\log n)$, esto nos da una complejidad total de $O(V \log V)$, menor a $O((V+E) \log V)$.

Tabla de resultados

Ejercicio	Resultado
1	Completo
2	Completo
3	Completo
4	Completo
5	Completo